`Owen Richmond

EECE4632

Febuary 12, 2026

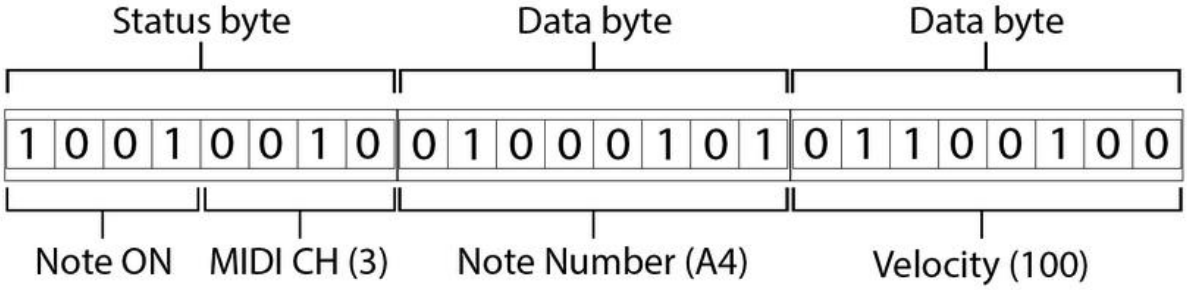
Project Proposal: FPGA-Based Audio Effects Processor

Abstract

I propose to design a real-time audio effects processor on the Zynq Ultrascale+ XCZU3EG, a guitar pedal using simplified midi as an input. I plan to implement a hardware-accelerated DSP pipeline in the programmable logic, focusing on 2-3 audio effects like distortion, delay, or tremolo. The PS will send simple MIDI-formatted note messages and control parameters while the PL generates basic tones and processes them at 48 kHz. My main goal is to learn FPGA DSP design methodology, with physical controls, audio output, and OLED display as potential stretch goals.

Inputs and Outputs

The effects processor will take simple MIDI-style messages as input - just the basic 3-byte format (status byte for Note ON/OFF, note number like A4, and velocity). I'll also have control parameters for each effect like gain, delay time, modulation rate, and enable/bypass flags. I'm planning to use AXI-Lite for control registers and possibly AXI-Stream for audio sample flow. The output will be processed audio samples (16-bit PCM at 48 kHz), plus status registers for debugging.



Simplest MIDI 1.0 “On” Note: Easy to Generate With Python!

Input and Output Sources

For development, I'll write Python code on the PS to send simple MIDI messages. As I've discovered while researching this project, MIDI contains hundreds of messages and variations. I believe spending so much time in python decoding this into another hundred flags wouldn’t be very fun. Instead I will use the basic message structure above as a convenient way

to specify notes. For example, a Note ON message would be three bytes: status (Note ON), note number (like 69 for A4), and velocity (amplitude). The PL will read these messages, generate a tone at the corresponding frequency, and then I will apply effects to it. My plan is to store audio samples in DDR memory as raw 16-bit PCM at 48 kHz and use AXI-Stream DMA for data transfer. The PS can read back processed audio for analysis after adding a WAV header to form a .wav file. There are plenty of Python libraries that make this achievable without straying too far from the core FPGA focus. This approach lets me focus on the FPGA DSP design without needing complex MIDI hardware or audio interfaces initially.

If time permits, I'd like to add physical controls using onboard switches and potentiometers, route audio to a codec via I2S for real-time playback, and maybe use the PMOD OLED display to show effect parameters or status.

Processing and Algorithms

I'm planning to explore 2-5 guitar pedal style effects. Some possibilities include:

Tremolo: Modulate the amplitude using a low-frequency oscillator. This seems like a good starting effect since it's simpler than the others.

Distortion/Overdrive: Apply waveshaping or clipping to create harmonic distortion. I'll need to figure out the right clipping function and gain scaling using fixed-point math.

Delay/Echo: Create time-delayed copies of the input using a circular buffer in BRAM. This will involve managing buffer pointers and implementing feedback for multiple echoes.

I might also look into chorus, flanger, or simple reverb depending on how the first effects go. But these top 3 are my first priority. The effects will be chained together in series, and I'll use fixed-point arithmetic throughout to meet the 48 kHz timing requirement.

Project Plan

I expect I'll need to adjust this plan as I run into challenges and learn more about Vitis and Vivado. While I am optimistic in the high level overview of what I plan to implement, the challenge of specific development milestones are uncertain. This is my rough idea of how the work will progress.

Phase 1: Get comfortable with the PS side - write Python code to generate simple MIDI messages and set up AXI-Stream DMA. Create an example “sound” with basic midi input from PS.

Phase 2: Implement one simple effect (probably tremolo) in Vitis HLS. Learn how to use HLS pragmas, package IP, and integrate it in Vivado. Test it against a Python reference.

Phase 3: Add a second effect (distortion or delay) and start chaining effects together. Figure out how to manage multiple IP blocks and parameter control.

Phase 4: Complete the core effects pipeline and do comprehensive testing with different note sequences and parameters.

Phase 5: Optimize timing and meet resource constraints.

Phase 6 (Stretch): If the core pipeline works well, explore adding physical controls with switches and potentiometers.

Phase 7 (Stretch): Implement I2S audio output to route processed audio to a codec for real-time playback.

Phase 8 (Stretch): Use the PMOD OLED display to show effect status, parameter values, or simple visualizations.

Phase 9 (Super Stretch): Use the Midikeyboard I have!

My Midi Keyboard (Nektar se25)

Evaluation

I'll verify that my effects work correctly by comparing the PL output to reference implementations in Python or MATLAB. I'll test with different note sequences, parameter sweeps, and recorded audio samples. Basically I can compare it to a regular DAW like Audacity, which i already have installed and am familiar with. The output should sound right and match expected DSP behavior, distortion should add harmonics, delay should create echoes at the right timing, tremolo should modulate amplitude smoothly.

Success means the effects run in real-time at 48 kHz, I can control parameters via AXI-Lite, the output sounds good and doesn't cause horrible screeching. But, most importantly, everything fits within the FPGA resources. I'll use Python with matplotlib to visualize waveforms and compare hardware vs software outputs, I think that would make for good visuals on the final report.

February 29th Deadline Deliverables

By February 29th, I plan to have:

- Python code to generate simple MIDI messages on the PS
- A basic tone generator or pass-through IP in HLS
- At least one working effect, ideally 2 (tremolo and distortion) implemented in HLS
- A Vivado block design with the IP integrated
- A bitstream and PYNQ overlay
- Python test code to send notes and control parameters
- Verification that the hardware output matches a software reference
- Documentation of my control registers

This will prove that the basic PS-PL interface works and give me a foundation to build the full effects chain.

References

Vitis HLS User Guide: https://docs.amd.com/r/2024.1-English/ug1399-vitis-hls

Python Library .WAV -> MP3: https://pypi.org/project/pyaudioconvert/

AXI Reference Guide:
https://docs.amd.com/r/2024.1-English/ug1399-vitis-hls/Customizing-AXI4-Stream-Interfaces

PYNQ Overlays Documentation:
https://pynq.readthedocs.io/en/v2.6.1/overlay_design_methodology/overlay_tutorial.html

Audio DSP Theory (CCRMA Stanford): https://ccrma.stanford.edu/~jos/pasp/

MIDI Explained (Background): https://learn.piratemidi.com/learn/lesson/1-midi-explained